

Track B – RBC Project: Concurrency and Transactions

Introduction

In Assignment 3, you implemented a backend service for a simplified banking platform using **FastAPI** and **SQLAlchemy**. The system supported basic banking operations such as retrieving accounts, performing transfers between accounts, viewing transaction history, and running analytics queries. One of the key APIs implemented in that assignment was the **money transfer endpoint**, which included several steps such as:

- validating both accounts
- verifying that accounts are active
- checking that the transfer amount is valid
- verifying that the sender has sufficient balance
- subtracting the amount from the sender
- adding the amount to the receiver
- creating a transfer record
- inserting ledger entries

For the purpose of Assignment 3, you were intentionally instructed **not to implement transaction management**. This allowed you to focus on building the backend logic and understanding how multiple tables interact within a real system.

However, in real-world banking systems this simplified implementation introduces a critical issue: **concurrency problems**.

In production environments, many users may perform banking operations simultaneously. If multiple transfer requests interact with the same account at the same time, the system may produce incorrect results unless proper transaction control is used. For example:

- Two transfer requests may read the same account balance simultaneously.
- Both may assume the balance is sufficient.
- Each request then performs its own update independently.

This can result in **incorrect account balances or inconsistent ledger entries**, which is unacceptable in financial systems.

To prevent these issues, backend systems rely on **database transactions and isolation mechanisms**. Transactions ensure that multiple database operations behave as a **single logical unit of work**, meaning either all changes succeed together or none of them are applied.

This assignment builds directly on your previous implementation. You will analyze how concurrency affects your transfer logic and update your system to make it reliable using **transactions and concurrency reasoning**.

How to Start

To complete this assignment, you must continue working with the **same project you built in Assignment 3**.

Locate your previous implementation of the **RBC Mini Banking Backend** and ensure that your application runs correctly using:

```
uvicorn main:app --reload
```

You will modify your existing code and answer several short conceptual questions based on the behavior of your system. You do **not need to rebuild the project**. The purpose of this assignment is to improve the **correctness and reliability of your existing API implementation**.

Part 1 – Concurrency Scenario Analysis

Consider the following situation in your banking system. Initial database state:

```
Account ID: 12
Balance: 100
```

Two transfer requests arrive at the same time.

Request A	Request B
<pre>POST /transfers from_account_id = 12 to_account_id = 18 amount = 80</pre>	<pre>POST /transfers from_account_id = 12 to_account_id = 25 amount = 50</pre>

Both requests reach the server at nearly the same time. Because the Week 3 implementation did not include transaction management, both requests may read the same account balance before either request updates it.

Questions

Create a file named `Part1_Answers.txt` in your `week4_assignment` git repository. Answer the following questions.

1. What incorrect balance could occur if both requests execute concurrently?
2. What type of problem does this represent (race condition, lost update, etc.)?
3. Why does this issue occur in the Week 3 implementation?

Your answer should be concise (approximately **4–6 sentences total**).

Part 2 – Implement Transaction Protection

You must now improve your **POST /transfers** endpoint to ensure that the transfer operation behaves as a **single atomic transaction**. If any step of the transfer fails, all previous operations must be **rolled back**.

For example:

- If balance validation fails
- If the debit operation fails
- If ledger insertion fails
- If any database error occurs

then **no partial updates should remain in the database**.

Requirements

Modify your transfer logic so that the following operations execute within **one database transaction**:

1. Validate both accounts exist
2. Validate that both accounts are active
3. Verify the transfer amount
4. Check that the sender has sufficient balance
5. Subtract the amount from the sender
6. Add the amount to the receiver
7. Insert the transfer record
8. Insert ledger entries
9. You may implement transaction handling using SQLAlchemy. Example pattern:

```
10. with db.begin():
11.     # perform database operations
12. or
13. try:
14.     db.begin()
15.
16.     # operations
17.
18.     db.commit()
19. except:
20.     db.rollback()
```

Your implementation must ensure that **either the entire transfer succeeds or no changes are applied to the database**. Put a copy of your updated implementation with some comments in your week4_assignment git repository

Part 3 – Isolation Level Reasoning

Transactions operate under different **isolation levels**, which define how concurrent transactions interact. Create a file named `Part3_Answers.txt` in your week4_assignment git repository. Answer the following questions.

1. Why would **READ UNCOMMITTED** be unsafe in a banking system?
2. What type of issue does **REPEATABLE READ** help prevent?
3. Why might **SERIALIZABLE** provide stronger guarantees but reduce system performance?

Each answer should be **2–3 sentences**.

Part 4 – Concurrency in Real Systems

Imagine that **100 users attempt transfers at the same time** through your API.

Answer the following questions in a file named `Part4_Answers.txt` in your week4_assignment repository.

1. How do transactions protect account balances during concurrent transfers?
2. What role does connection pooling play when many users access the API simultaneously?
3. What risks might occur if the system had **no transaction management and no connection pooling**?

Provide short explanations (2–4 sentences each).

Submission Instructions

Your submission must include:

- Your updated backend implementation
- The following files in your git repository:

```
Part1_Answers.txt
Part2.py
Part3_Answers.txt
Part4_Answers.txt
```

Ensure that your application still runs successfully using the following command and then share your git repository link with me on Brightspace under assignment 4 or through emailing me.

```
uvicorn main:app --reload
```